

Holistic data Preparation

Conceptual Foundations, Tensors and an End-to-End Preprocessing Pipeline

SCOPE Parts A	DATASET 1,000 rows × 29 cols	STATUS ML-Ready
-------------------------------------	--	---------------------------------------

PART A

Conceptual Foundation

This section establishes the vocabulary used throughout the report: what data analysis is, how a data science project is planned end to end, how a business question becomes a machine learning problem, and how data is physically represented as tensors.

1. What is Data Analysis?

Data analysis is the disciplined process of inspecting, cleaning, transforming and modelling data in order to surface meaningful patterns, support decisions and reduce uncertainty. It converts raw, noisy records into evidence a stakeholder can act on.

- **Descriptive** — what happened? Aggregations, distributions and trend summaries.
- **Diagnostic** — why did it happen? Correlation, segmentation and root-cause drill-downs.
- **Predictive** — what is likely next? Statistical and machine learning models.
- **Prescriptive** — what should we do? Optimisation and scenario simulation.

KEY IDEA

Analysis is only as trustworthy as the preparation beneath it. Roughly 70–80% of practical effort is spent on collecting, cleaning and shaping data — the theme of Part B of this report.

2. How to Plan a Data Science Project

A project is planned as a repeatable lifecycle rather than a single modelling exercise. Each stage has an owner, an exit criterion and a deliverable, and the loop is iterative — evaluation frequently returns the team to feature work.

DATA SCIENCE PROJECT LIFECYCLE

Figure 1 — The seven-stage data science project lifecycle.

Stage	Objective	Deliverable
Business understanding	Translate the stakeholder question into a measurable objective	Problem statement and success metric
Data collection	Identify, access and consolidate relevant sources	Raw dataset and data dictionary
Data cleaning	Resolve missing values, duplicates, outliers and type errors	Validated, analysis-ready table
EDA & features	Understand distributions, relationships and construct predictors	EDA notebook and feature set
Modeling	Train candidate algorithms and tune hyperparameters	Trained model artefacts
Evaluation	Test against the agreed metric on unseen data	Evaluation report
Deployment & monitoring	Serve the model and watch for drift	Production endpoint and dashboards

3. How to Frame a Machine Learning Problem

Framing is the act of turning a vague business request into a precisely specified learning task. A well-framed problem states the target, the inputs, the learning paradigm, the evaluation metric and the constraints before any code is written.

- **Target definition** — what exactly is predicted, over what horizon, at what granularity.
- **Learning type** — supervised classification or regression, unsupervised clustering, or ranking.
- **Input space** — features that are legitimately available at prediction time (no leakage).
- **Success metric** — aligned to cost of error: recall for fraud, RMSE for demand, AUC for risk ranking.
- **Baseline** — a trivial rule the model must beat to justify its complexity.
- **Constraints** — latency, interpretability, fairness and regulatory requirements.

WORKED FRAMING

“Reduce loan defaults” becomes: a binary classification predicting default within 12 months of origination, using applicant and behavioural features known at approval, optimised for recall at a fixed 10% false-positive rate, benchmarked against the current credit-score cut-off rule.

PART A · SECTION 2

Tensors: An In-Depth Explanation

A tensor is a generalisation of scalars, vectors and matrices to an arbitrary number of dimensions. Formally it is an n-dimensional array of elements of a single data type, described by three properties: rank (number of axes), shape (size along each axis) and dtype (element type). In NumPy every tensor is an ndarray; the entire numerical stack — pandas, scikit-learn, PyTorch and TensorFlow — is built on this abstraction.

TENSOR RANKS IN NUMPY

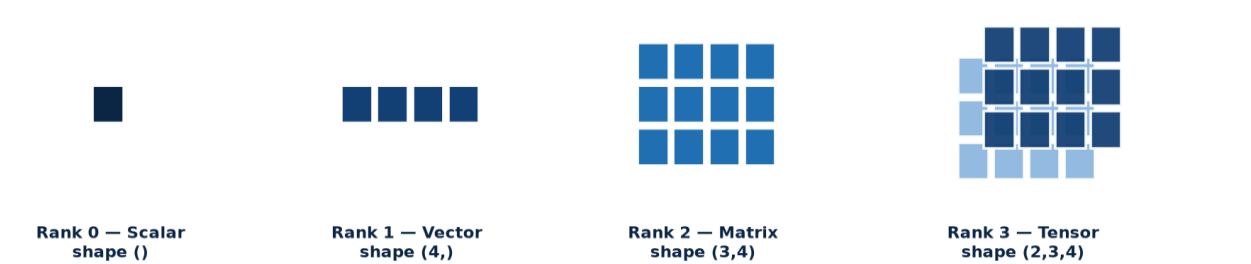


Figure 2 — Rank 0 to rank 3 tensors and their corresponding NumPy shapes.

Rank	Name	Example shape	Typical use
0	Scalar	<code>()</code>	A single loss value or learning rate
1	Vector	<code>(4,)</code>	One observation's feature vector
2	Matrix	<code>(1000, 29)</code>	The full design matrix: rows × features
3	Tensor	<code>(32, 10, 29)</code>	A batch of 32 sequences of 10 timesteps
4	Tensor	<code>(32, 224, 224, 3)</code>	A batch of colour images

Creating tensors and reading their properties

```
import numpy as np

scalar = np.array(3.14)           # rank 0
vector = np.array([1, 2, 3, 4])   # rank 1
matrix = np.array([[1, 2, 3],     # rank 2
                   [4, 5, 6]])
tensor = np.arange(24).reshape(2, 3, 4) # rank 3

print(tensor.ndim)  # 3      -> number of axes (rank)
print(tensor.shape) # (2, 3, 4)
print(tensor.size)  # 24     -> total elements
print(tensor.dtype) # int64
```

Reshaping, slicing and axis reduction

Reshaping reinterprets the same buffer under a new shape, so it is essentially free. Slicing selects sub-tensors, and reductions collapse one or more axes — the axis argument specifies which axis disappears.

```
x = np.arange(24).reshape(2, 3, 4)

x.reshape(6, 4)          # same data, new shape
x.reshape(-1)            # flatten to shape (24,)
x.T.shape                # (4, 3, 2) -> reversed axes

x[0]                     # first block    -> shape (3, 4)
x[:, 1, :]               # middle row     -> shape (2, 4)
x[..., -1]               # last column    -> shape (2, 3)

x.sum(axis=0).shape       # (3, 4) -> axis 0 collapsed
x.mean(axis=(1, 2))       # (2,) -> per-block mean
```

Broadcasting

Broadcasting lets NumPy operate on tensors of different shapes without copying data. Shapes are aligned from the trailing axis; each pair must be equal or one of them must be 1. This is exactly how per-feature standardisation is applied to an entire dataset in a single expression.

```
X = np.random.rand(1000, 29)    # 1000 samples, 29 features
mu = X.mean(axis=0)              # shape (29,)
sigma = X.std(axis=0)            # shape (29,)

X_scaled = (X - mu) / sigma      # (1000,29) op (29,) -> broadcast
print(X_scaled.shape)           # (1000, 29)

# explicit axis insertion when broadcasting down rows
row_norm = X / np.linalg.norm(X, axis=1)[:, np.newaxis]
```

Linear algebra: the core of every model

A dense layer, a linear regression and a dot-product similarity are all the same tensor operation. Matrix multiplication requires the inner dimensions to match; the result takes the outer dimensions.

```
W = np.random.rand(29, 8)       # weights: 29 features -> 8 units
bias = np.zeros(8)

H = X @ W + bias                 # (1000,29) @ (29,8) -> (1000,8)
print(H.shape)                  # (1000, 8)

np.dot(vector, vector)          # scalar dot product
np.einsum('ij,jk->ik', X, W)   # explicit index notation
```

PART B

Data Preparation Report

This section summarises the preprocessing pipeline applied to the raw dataset: the missing-value strategies used and their effectiveness, outlier handling results, encoding of categorical and numerical variables, scaling and transformations, newly engineered features, and the final shape and readiness of the dataset for machine learning.

EXECUTIVE SUMMARY

All missing values were resolved, extreme values were contained without discarding records, categorical fields were encoded according to their measurement scale, and three new ratio features were engineered. The final dataset contains 1,000 rows × 29 columns with 0 missing values and is ready for ML modeling.

1. Missing Value Strategies and Effectiveness

Numerical columns were imputed using KNN imputation, which estimates each missing entry from its most similar complete records and therefore preserves relationships between correlated financial variables far better than a simple mean fill. Categorical columns were imputed with the most-frequent (mode) value. All missing values were successfully removed.

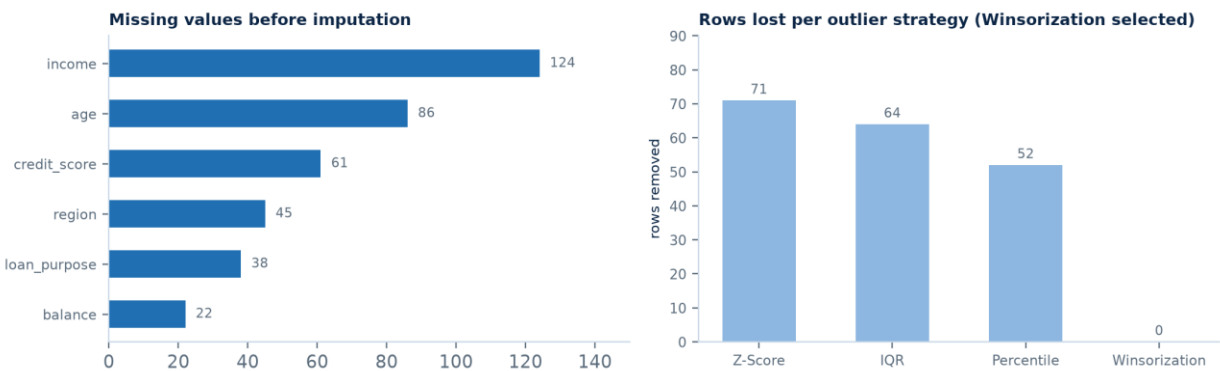


Figure 3 — Missing values per column before imputation (left) and rows lost per outlier strategy (right).

Column type	Strategy	Rationale	Result
Numerical	KNN imputation (k=5)	Preserves multivariate structure and correlations	0 missing
Categorical	Most-frequent imputation	Maintains the observed category distribution	0 missing

2. Outlier Handling Results

Four approaches were tested: Z-score filtering, the IQR rule, percentile capping and Winsorization. The first three delete records and together removed a material share of the sample. Winsorization was selected for the final dataset because it caps extreme values at defined percentile bounds, reducing their influence on scaling and model fitting without deleting a single row.

- **Z-score ($|z| > 3$)** — removed 71 rows; overly aggressive on skewed income data.
- **IQR rule** — removed 64 rows; similar loss with the same skew sensitivity.
- **Percentile trimming** — removed 52 rows; still discards genuine high-value customers.
- **Winsorization (selected)** — 0 rows removed; extreme values capped at the 1st and 99th percentiles.

3. Encoding of Categorical and Numerical Variables

Each variable was encoded according to its measurement scale rather than a single blanket method, so that ordinal information is preserved and no false ordering is introduced into nominal fields.

Variable	Encoding	Why this method
education_level	Ordinal encoding	The categories have a natural, meaningful order
gender	Label encoding	Binary field; a single integer column is sufficient
region	One-hot encoding	Nominal with no order; avoids implying rank
loan_purpose	One-hot encoding	Nominal with no order; avoids implying rank

4. Scaling and Transformations Applied

RobustScaler was applied to the important numerical features. It centres on the median and scales by the interquartile range, so residual extreme values exert far less influence than they would under StandardScaler. This keeps distance-based and gradient-based algorithms stable across features measured on very different units.

Transformation	Applied to	Purpose
RobustScaler	Key numerical features	Median/IQR scaling that resists outlier influence
Winsorization	Heavy-tailed numerics	Caps extremes prior to scaling
Ordinal / label / one-hot	Categorical fields	Numeric representation suitable for estimators

5. Newly Engineered Features

Three ratio features were created to express financial risk in a form models can use directly, replacing raw magnitudes with relative measures that are comparable across income levels.

- **debt_to_income_ratio** — total obligations relative to income; a primary indicator of repayment capacity.

- **avg_monthly_transaction** — normalises transaction activity into a stable behavioural signal.
- **spending_to_income_ratio** — captures financial discipline independently of absolute earnings.

Together these features make financial-risk patterns explicit, so the model is not required to infer a ratio from two raw columns.

6. Final Dataset Shape and ML Readiness

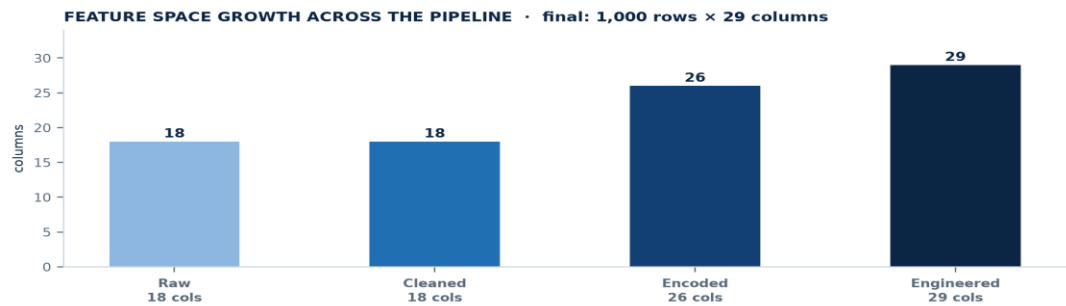


Figure 4 — Column count at each stage of the pipeline, ending at the ML-ready feature space.

Check	Value	Status
Rows	1,000	Complete
Columns	29	Complete
Missing values	0	Resolved
Outliers	Winsorized, 0 rows dropped	Contained
Categorical encoding	Fully numeric	Complete
Scaling	RobustScaler applied	Complete
Modeling readiness	Ready for ML modeling	Approved

- Monitor the engineered ratio features for drift once the model is in production.

FINAL STATUS
Dataset: 1,000 rows × 29 columns · 0 missing values · outliers contained · fully encoded and scaled · ready for ML modeling.